Demanda, Provisionamento e Desempenho

Aplicações desenvolvidas

A ideia da aplicação é identificar cadastros duplicados em arquivos CSV e eliminar as ocorrências anteriores, mantendo a última encontrada de cada ID. Para fazer essa comparação, os registros únicos permanecem em memória durante o processamento. Assim, o envio simultâneo de arquivos aumenta a necessidade de RAM. O backend foi desenvolvido em Python 3.10 com FastAPI e retorna as contagens de registros e o tempo de processamento.

Foi usado o mesmo lote de quatro CSVs (99,15 MiB), com 96000 registros e 88000 IDs únicos, em todas as requisições.

Configuração da VM e Deploy da Aplicação

Por dificuldades de preparar o ambiente com VirtualBox no computador disponível, os experimentos foram realizados localmente com Docker. Isso permitiu controlar RAM e CPU e repetir os testes com a mesma imagem da aplicação, construída pelo Dockerfile.

O hospedeiro foi um Mac (Apple M5, 16 GiB de RAM). A VM Linux interna do Docker Desktop dispunha de 7,75 GiB e 10 CPUs lógicas. Em cada container, aplicaram-se limites de 500 MiB, 1 GiB, 2 GiB ou 4 GiB de RAM e quota fixa de seis CPUs, com swap desabilitado e seis workers. O Locust executou no hospedeiro, acessando a porta publicada da API.

Teste de estresse da aplicação selecionada

O teste buscou observar o que acontece quando mais clientes usam a aplicação ao mesmo tempo e quando ela dispõe de menos memória. Para cada limite de RAM, simularam-se 1, 2, 4 e 8 usuários simultâneos. Cada combinação foi repetida três vezes, totalizando 48 execuções.

Cada usuário virtual enviava o lote de CSVs, aguardava a resposta e iniciava outro envio. Cada execução começava com um container novo. Depois que todos os usuários estavam ativos, houve 10 segundos de aquecimento e, em seguida, 60 segundos de medição. CPU e memória foram registradas a cada meio segundo; entre execuções, aguardaram-se cinco segundos, alternando as configurações testadas.

A vazão mostra quantos envios foram concluídos corretamente por segundo, e a latência informa o tempo de resposta. O p95 é o tempo que pelo menos 95% das respostas válidas analisadas não ultrapassaram. Também se mediram o pico de memória e o uso de CPU em relação ao limite de seis CPUs.

Os tempos de resposta consideram apenas envios bem-sucedidos que começaram e terminaram durante a medição. Envios ainda em andamento ao final foram registrados separadamente. Falhas de atendimento e encerramentos por falta de memória (OOM) também foram registrados; os contadores OOM incluem o aquecimento.

Na tabela, os valores são médias das três repetições, inclusive o p95. O símbolo ± mostra o desvio padrão, que indica quanto os valores variaram entre as repetições.

Os 48 testes concluíram seus 60 segundos de medição. Em 9 execuções faltaram amostras do monitor: essas coletas ficaram fora das médias de CPU/memória, mas os dados de requisições foram mantidos. Três repetições não estabelecem significância estatística.

Comparação na maior demanda: oito usuários simultâneos.

RAM	Vazão req/s	Latência média s	p95 s	Falhas %	Testes com OOM
500 MiB	0,09 ± 0,03	5,83 ± 0,61	8,61 ± 1,10	94,4 ± 1,2	3/3
1 GiB	0,88 ± 0,13	5,00 ± 0,30	8,08 ± 0,87	40,2 ± 4,7	3/3
2 GiB	1,40 ± 0,06	5,71 ± 0,10	11,75 ± 0,42	0,0 ± 0,0	0/3
4 GiB	1,67 ± 0,08	4,78 ± 0,10	6,84 ± 1,36	0,0 ± 0,0	0/3

Resultados e interpretação

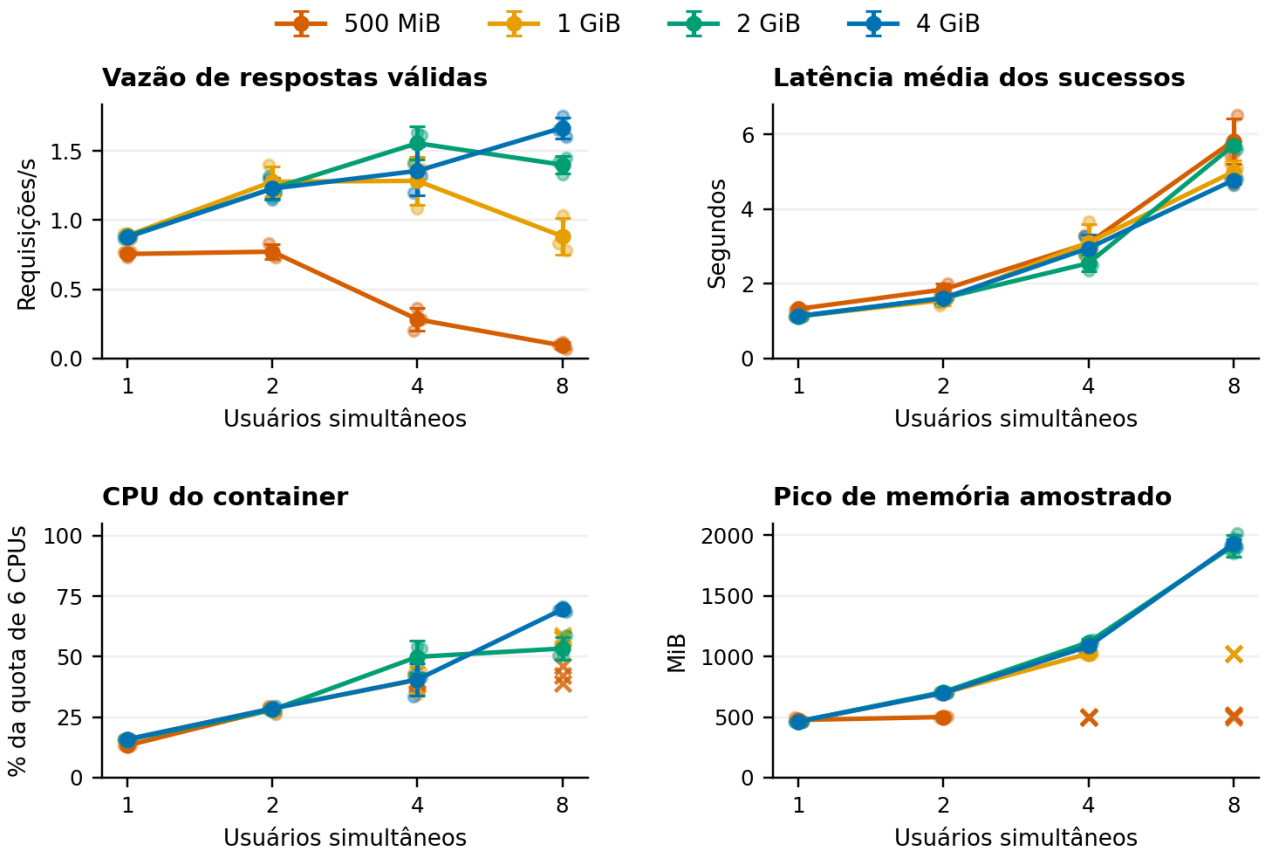


Figura 1. Médias ± desvio padrão; pontos claros mostram repetições. Cruzes indicam coletas de recursos com lacunas, excluídas das médias de CPU/memória.

Com 500 MiB, houve OOM a partir de dois usuários; com 1 GiB, em oito. Em 2 e 4 GiB, não houve falhas ou OOM até oito usuários. Ampliar RAM evitou mortes por falta de memória nos níveis testados, mas não elevou a vazão em todas as combinações de demanda.

Em 4 GiB, passar de um para oito usuários elevou a vazão de 0,88 para 1,67 req/s, e a latência de 1,14 para 4,78 s. O pico de memória passou de 464 para 1930 MiB; a CPU média com oito usuários foi 69,5% da quota.

Conclusão: maior demanda aumentou a latência e o consumo de recursos; RAM insuficiente causou falhas e queda da vazão válida. Os resultados valem para este hardware e dataset; parsing, GIL, upload e arquivos temporários também influenciam o desempenho. Latências consideram somente sucessos completos e devem ser lidas junto de falhas e pendentes.

Dados completos: results/docker-experiments-20261014T174435803277Z. Tabelas por cenário: output/analysis/comparison.csv.

Por que usar seis CPUs e seis workers?

Antes dos testes de RAM, foram comparadas duas configurações de CPU/workers, ambas com 4 GiB, oito usuários e três cargas de 30 segundos. Essa calibração orientou a escolha dos recursos que permaneceriam fixos nos 48 experimentos.

Configuração	Vazão média ± desvio padrão (req/s)	CPU média (% da quota)
4 CPUs / 4 workers	1,62 ± 0,14	75,44
6 CPUs / 6 workers	1,36 ± 0,12	52,63

Optou-se por seis CPUs e seis workers para ampliar a margem em relação à quota global de CPU. A comparação não demonstrou ganho de vazão: CPU e workers variaram juntos e as configurações foram executadas em sequência. Portanto, não se conclui que seis CPUs causem pior desempenho nem que a RAM seja o único gargalo. Os resultados da calibração foram analisados separadamente dos experimentos finais.